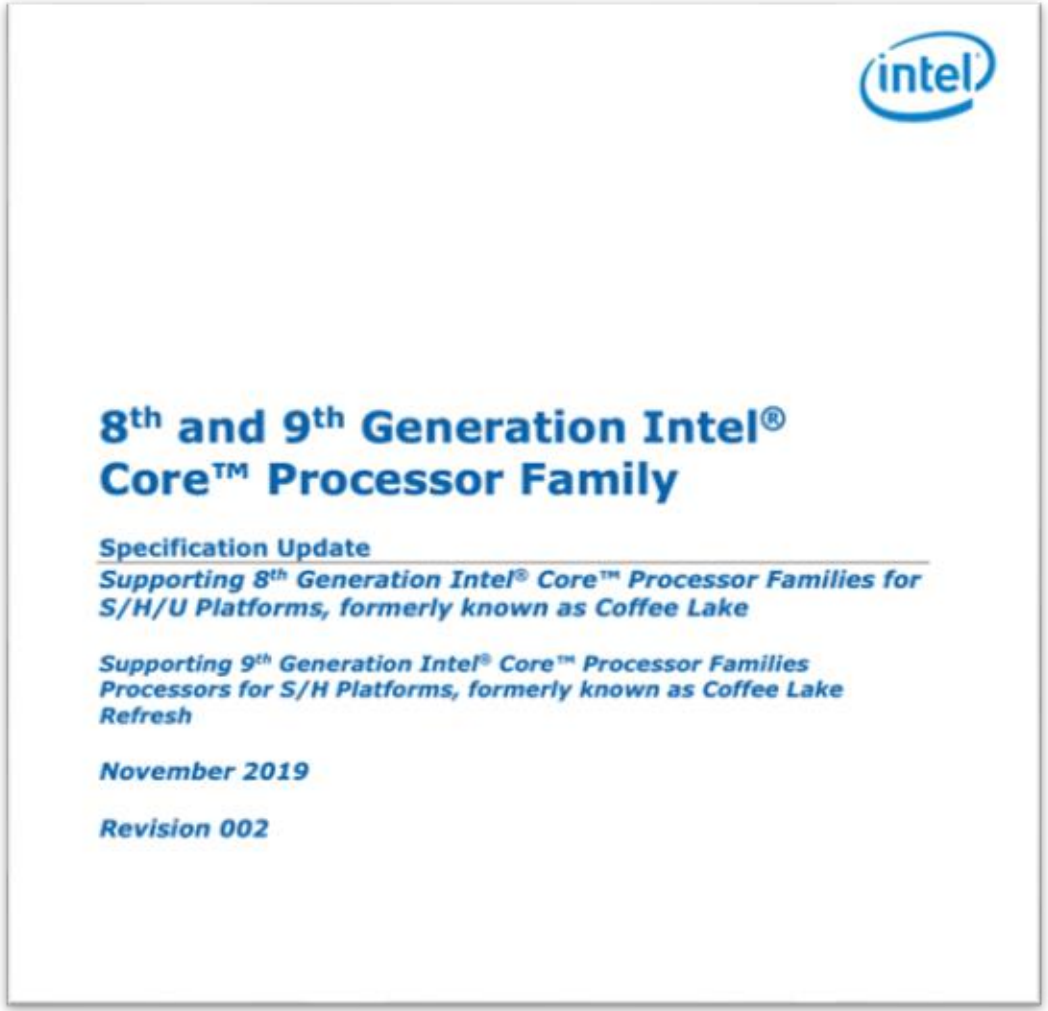
Hardware Bugs and Fuzzing

Mengjia Yan

Spring 2026

sysret slides credit: Will Liu (MIT)

What is Errata?



8th and 9th Generation Intel® Core™ Processor Family

Specification Update

Supporting 8th Generation Intel® Core™ Processor Families for S/H/U Platforms, formerly known as Coffee Lake

Supporting 9th Generation Intel® Core™ Processor Families Processors for S/H Platforms, formerly known as Coffee Lake Refresh

November 2019

Revision 002

It is a compilation of device and document errata and specification clarifications and changes, which is intended for hardware system manufacturers and for software developers of applications, operating system, and tools.

Errata are design defects or errors. Errata may cause the processor's behavior to deviate from published specifications. Hardware and software designed to be used with any given stepping must assume that all errata documented for that stepping are present on all devices.

https://www.intel.com/content/www/us/en/products/docs/processors/core/8th-gen-core-spec-update.html

Errata Table Example

3.2 Errata Summary Information

Table 4-3. Errata Summary Table

ID	Processor Line / Stepping			Title
	S	H	U	
001				
002				
003				
004				

001	**Reported Memory Type May Not Be Used to Access the VMCS and Referenced Data Structures**
Problem	Bits 53:50 of the IA32_VMX_BASIC MSR report the memory type that the processor uses to access the VMCS and data structures referenced by pointers in the VMCS. Due to this erratum, a VMX access to the VMCS or referenced data structures will instead use the memory type that the memory-type range registers (MTRRs) specify for the physical address of the access.
Implication	Bits 53:50 of the IA32_VMX_BASIC MSR report that the write-back (WB) memory type will be used, but the processor may use a different memory type.
Workaround	Software should ensure that the VMCS and referenced data structures are located at physical addresses that are mapped to WB memory type by the MTRRs.
Status	For the steppings affected, refer the Summary Table of Changes.

More Errata

Occasionally, AMD identifies product errata that cause the processor to deviate from published specifications. Descriptions of identified product errata are designed to assist system and software designers in using the processors described in this revision guide. This revision guide may be updated periodically.

AMD

Revision Guide for AMD Family 10h Processors

Publication # 41322 Revision: 3.92
Issue Date: March 2012

Advanced Micro Devices

298 L2 Eviction May Occur During Processor Operation To Set Accessed or Dirty Bit

Description

The processor operation to change the accessed or dirty bits of a page translation table entry in the L2 from 0b to 1b may not be atomic. A small window of time exists where other cached operations may cause the stale page translation table entry to be installed in the L3 before the modified copy is returned to the L2.

In addition, if a probe for this cache line occurs during this window of time, the processor may not set the accessed or dirty bit and may corrupt data for an unrelated cached operation.

Potential Effect on System

One or more of the following events may occur:

- Machine check for an L3 protocol error. The MC4 status register (MSR0000_0410) is B2000000_000B0C0Fh or BA000000_000B0C0Fh. The MC4 address register (MSR0000_0412) is 26h.
- Loss of coherency on a cache line containing a page translation table entry.
- Data corruption.

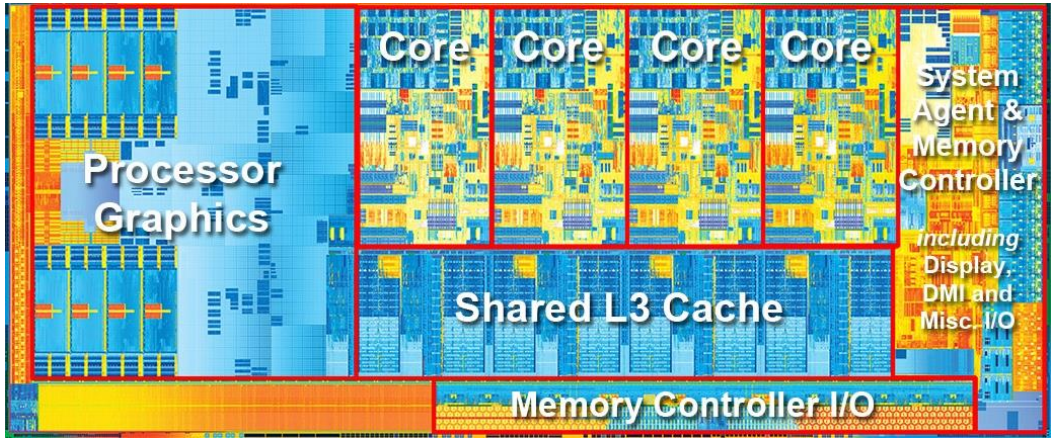
Suggested Workaround

BIOS should set MSRC001_0015[3] (HWCR[TlbCacheDis]) to 1b and MSRC001_1023[1] to 1b.

In a multiprocessor platform, the workaround above should be applied to all processors regardless of

https://www.amd.com/system/files/TechDocs/41322_10h_

Errata Statistics



The Core-i7 processor with integrated graphics card in 2012 with 1,400M Transistors

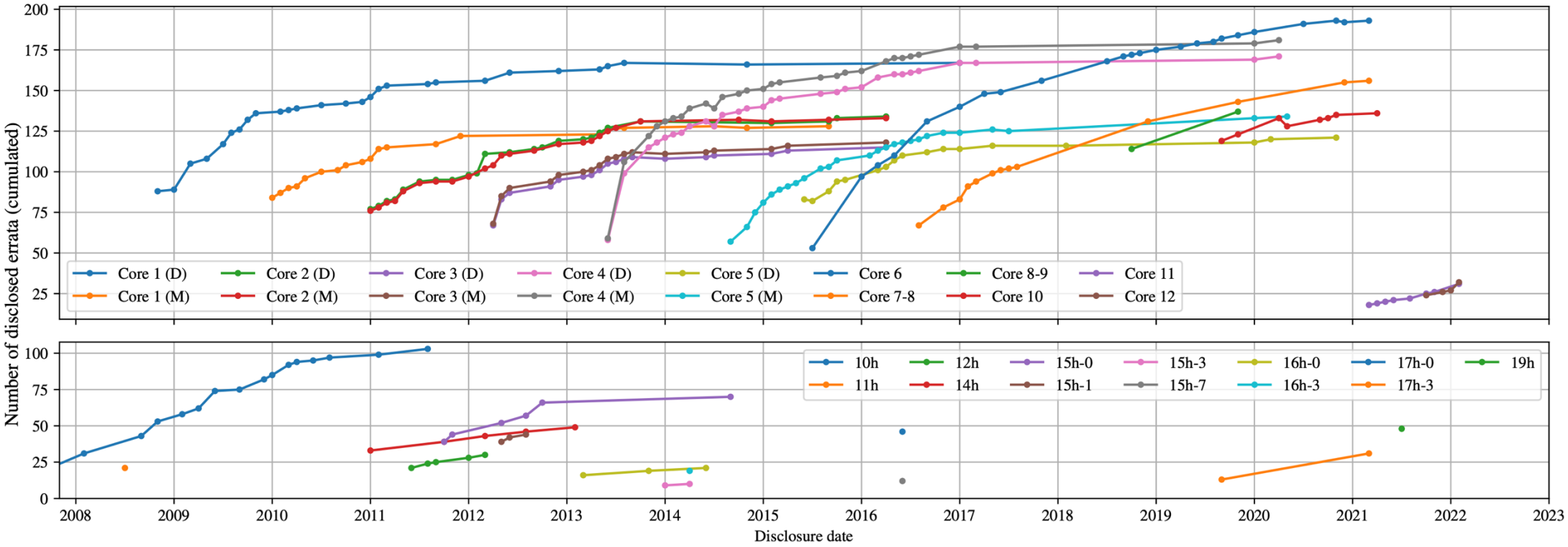
4 years; 136 errata; 3 bugs/month

A **4-fold increase** in bugs in Intel processor designs **per generation**. Approximately 8000 bugs designed into the Pentium 4 ('Willamette')

from *https://www.cl.cam.ac.uk/~jrh13/slides/nijmegen-21jun02/slides.pdf*

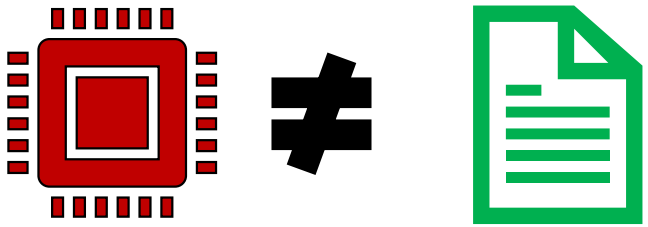
SPECS: A Lightweight Runtime Mechanism for Protecting Software from Security-Critical Processor Bugs; Hicks et al; ASPLOS'15
https://github.com/impedimentToProgress/specs

More Recent Errata Statistics

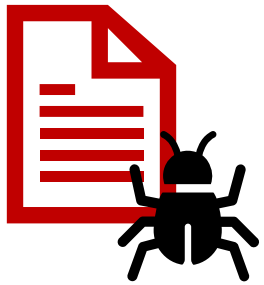


Disclosure dates of Intel Core errata (top) and AMD CPU errata (bottom).

Outline



Implementation does not match specification (Errata)



Bugs in the specification

Underspecification

- How to discover hardware bugs?
  - Manual efforts, testing
  - Fuzzing
  - Formal verification (next lecture)

Bug #1: Pentium FDIV Bug

- What is the specification for floating-point computation?
  - Floating is encoded as $(1 + f) \times 2^e, 0 \leq f < 1, e \in Z$
  - Example: $1/10 = 1.9999 \ldots 9a \times 2^{-4}$ (in hexadecimal)
  - We always have errors when doing floating-point computation, because we have limited number of bits for each floating number
- The specification allows error to occur after bit x

	Single precision	Double precision	Extended precision
Word size in bits	32	64	80
Bits for f	23	52	63
Bits for e	8	11	15
Relative accuracy	$2^{-23} \approx 1.2 \cdot 10^{-7}$	$2^{-52} \approx 2.2 \cdot 10^{-16}$	$2^{-63} \approx 1.1 \cdot 10^{-19}$
Approximate range	$2^{\pm 127} \approx 10^{\pm 38}$	$2^{\pm 1023} \approx 10^{\pm 308}$	$2^{\pm 16383} \approx 10^{\pm 4964}$

The Pentium FDIV bug: see errors much earlier than the expected x bits

The computational aspects of the Pentium affairs. Coe et al. IEEE 1995 https://people.cs.vt.edu/~naren/Courses/CS3414/assignments/pentium.pdf

The Discovery Process #1: Nicely's Prime

- Thomas Nicely, a mathematics professor, tried to compute reciprocal of prime numbers: $p = 824,633,702,441$
- The correct result:

$$1/p = 1.212659629408667 \times 10^{-12}$$

- But the new Pentium processor using **double precision** gives:

$$1/p = 1.212659624891158 \times 10^{-12}$$

- Took him four months to confirm the problem was NOT in his program -> math libraries -> compilers -> operating system, but in the hardware

Differ after the 9th digit

Any other numbers …?

The Discovery Process #2: Kaiser's List

- Andreas Kaiser, a computer consultant
  - Generate $25\ billion$ random integers and checked the accuracy of the computed reciprocals. 23 are incorrect.

3221224323	=	1.7ffff70600000	$\cdot\ 2^{31}$
12884897291	=	1.7ffff70580000	$\cdot\ 2^{33}$
206158356633	=	1.7ffff704c8000	$\cdot\ 2^{37}$
824633702441	=	1.7fffff7052000	$\cdot\ 2^{39}$
1443107810341	=	1.4fffedac25000	$\cdot\ 2^{40}$
6597069619549	=	1.7fffff7057400	$\cdot\ 2^{42}$
9895574626641	=	1.1fffc6bc2a200	$\cdot\ 2^{43}$
13194134824767	=	1.7ffff704e7e00	$\cdot\ 2^{43}$
26388269649885	=	1.7ffff704fdd00	$\cdot\ 2^{44}$
52776539295213	=	1.7ffff7046f680	$\cdot\ 2^{45}$

Patterns?

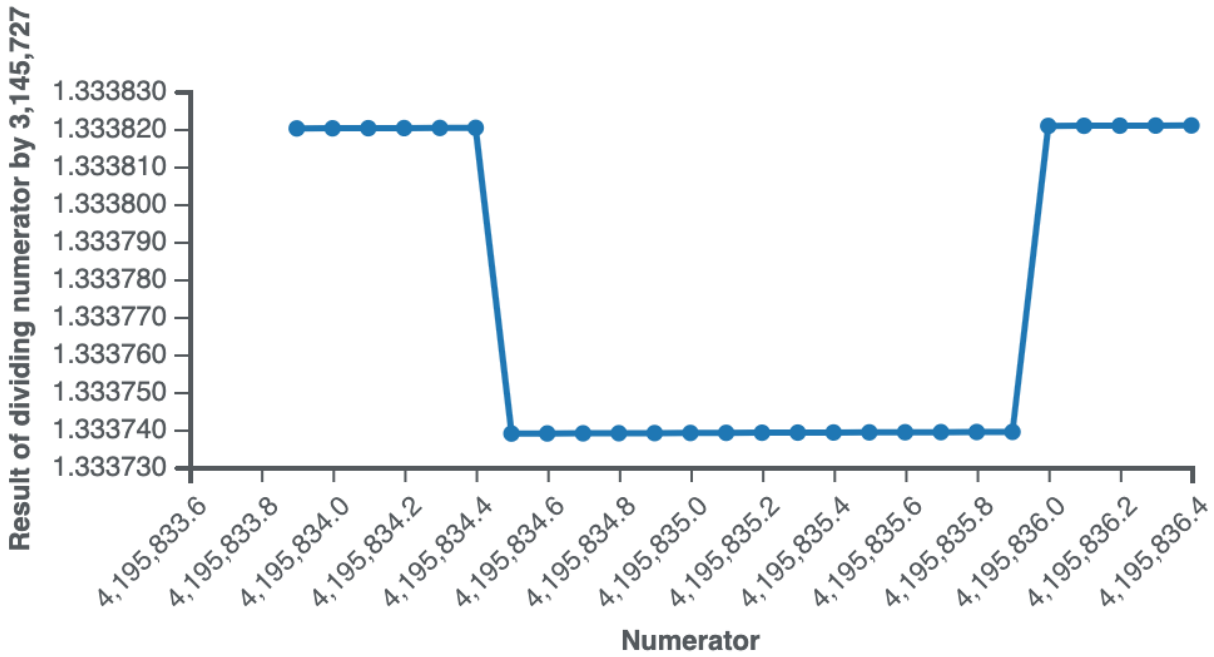
- Many are started with $1.7fffff$
- In another word, the first 20 bits after the leading bit have to be a single zero, followed by at least 19 ones

Pentium computed the reciprocals of these numbers inaccurately

The Discovery Process #3: Coe's Ratio

- Tim Coe, electrical engineer, has designed floating-point chips
  - $\frac{4{,}195{,}835}{3{,}145{,}727} = 1.33382044\ldots$ (correct) $1.33373906\ldots$ (Pentium)

Differ after the 4th digit



The erorrs involve q/p where q and p's bit patterns conspire to excite the bug at an early stage in the division.

Bug Explanation: FDIV

- Shift-and-subtract

```
                1.333?
3145727 ) 4195835
          3145727
          --------
          10501080
           9437181
           --------
           10638990
            9437181
            --------
            12018090
             9437181
             --------
             25809090
             ????????
```

How to choose quotient as a human being?

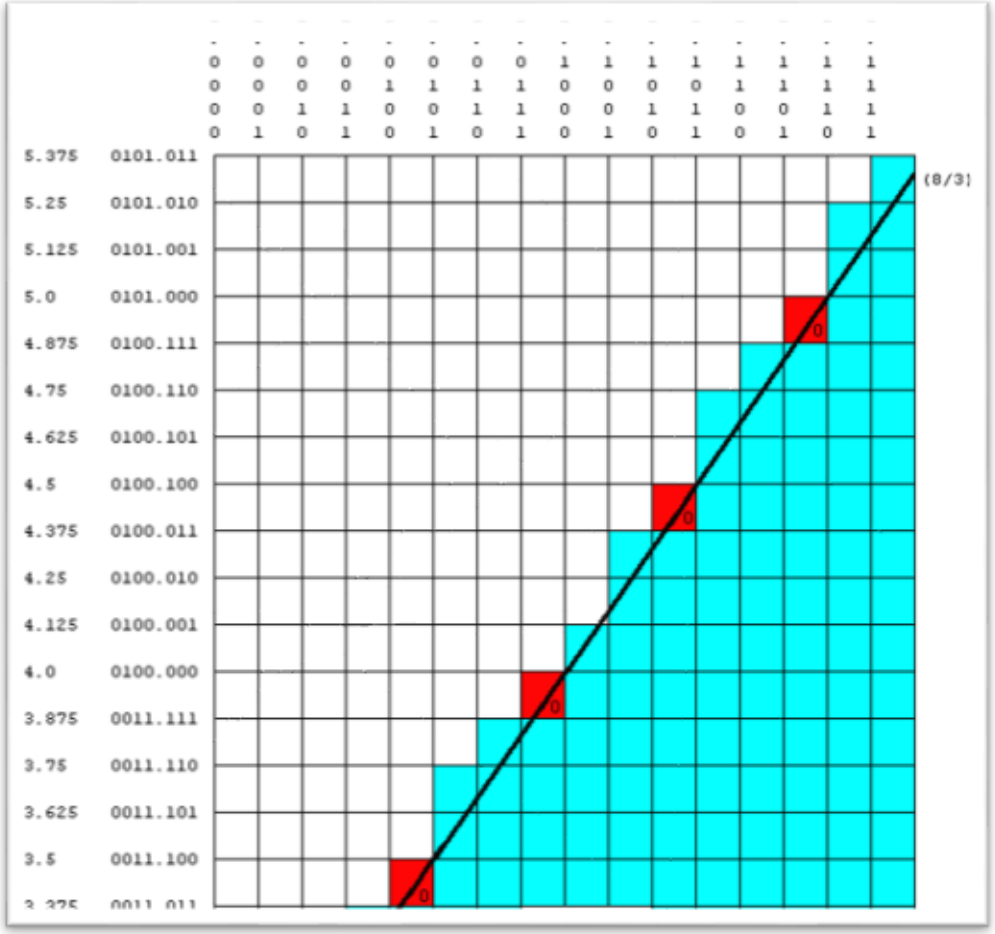
A combination of trial and error, experience, pattern matching and luck.

- Old processors: choose quotient from $0, 1$
- Faster Sweeney, Robertson, and Tocher (SRT) algorithm Radix-4:
  - Choose quotient from $0, +1, -1, +2, -2$;
  - If the current quotient is incorrectly chosen, we can recover it from the next iteration
  - Guess the quotient based on the first few digits => use a 2D table to lookup

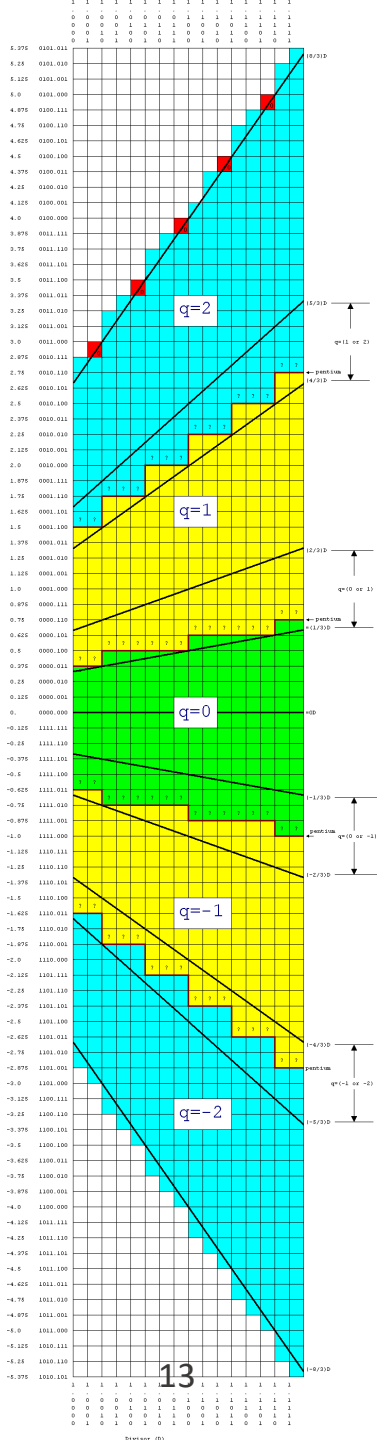
Bug Explanation: SRT Table

first 5 bits of the divisor

first 7 bits of the remainder



- 2048 cells in total
- 1066 cells in use
- **5 cells are not initialized**



How Frequently the bug can be triggered?

- Intel: an average spreadsheet user could encounter this flaw once in every 27,000 years, assuming 1,000 divisions per day.
- IBM: suspended sales of Pentium-based models and said it is as many as 20 mistakes per day.
- Who actually got affected?
  - Normal users?
  - Wall street? Financial pre-diction programs? Did the Pentium bug flip a trading decision from buy to hold to sell?
  - Difficult to calibrate

Consequences/Impacts

- Intel's bad responses
  - Conditional replacement (customers need to claim they do get influenced by the bug) → disastrous press
  - No-questions-asked replacement → $475M cost in 1994, 10% replacements

- Potential long-term impact:
  - Random test is not be a good idea. Exhaustive test has scalability problem.
  - A marked increase in the use of formal verification and number theory in hardware design

Some humor for you:

Q: How many Pentium designers does it take to screw in a light bulb?
A: 1.99904274017, but that's close enough for non-technical people.

Q: What do you get when you cross a Pentium PC with a research grant?
A: A mad scientist.

Do you think it bothers x86 users that the 486 is a functional upgrade to the Pentium?

In response to the Pentium bug, PowerMac officials have announced that they will be adding the control panel "Pentium Switcher" that allows users to decide whether the PowerMac should emulate pre-Pentium or post-Pentium FDIV behaviour.

TOP TEN NEW INTEL SLOGANS FOR THE PENTIUM

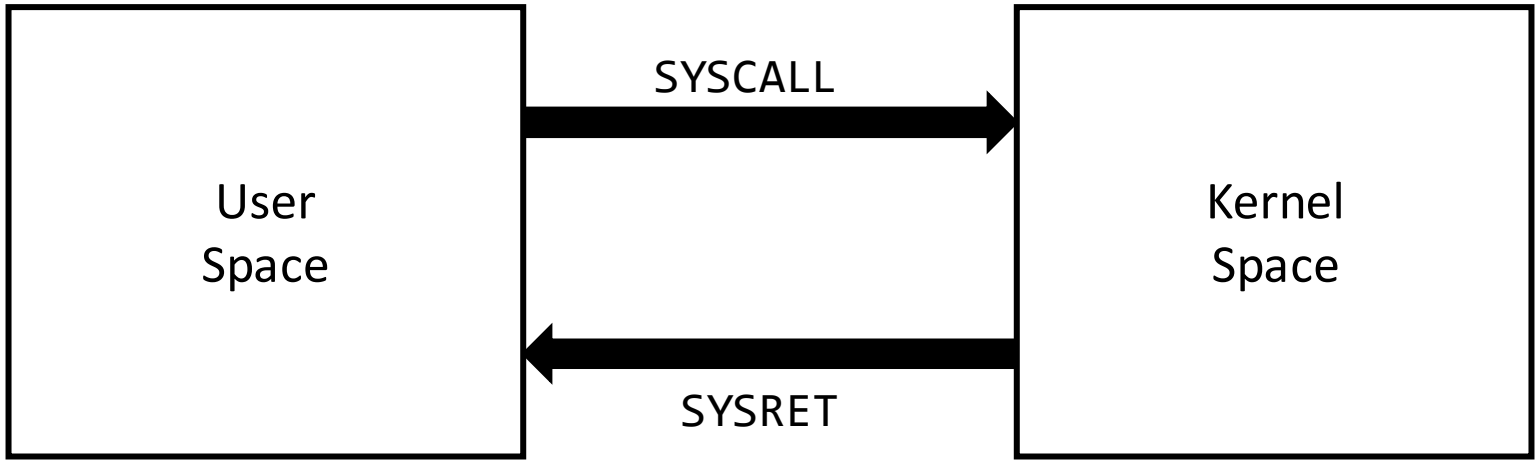
9.9999973251 It's a FLAW, Dammit, not a Bug
8.9999163362 It's Close Enough, We Say So
7.9999414610 Nearly 300 Correct Opcodes
6.9999831538 You Don't Need to Know What's Inside
5.9999835137 Redefining the PC--and Mathematics As Well
4.9999999021 We Fixed It, Really
3.9998245917 Division Considered Harmful
2.9991523619 Why Do You Think They Call It *Floating* Point?
1.9999103517 We're Looking for a Few Good Flaws
0.9999999998 The Errata Inside

http://davefaq.com/Opinions/Stupid/Pentium.html#glitch

Bug #2: A SYSRET Bug

64-bit x86 instruction set: AMD64, Intel 64

SYSCALL
- HW transits from user mode to kernel mode
- Save the userspace next-PC to the RCX register
- Jump to a kernel syscall entry point



SYSRET
- HW transits from kernel mode to user mode
- Restore the userspace next-PC from the RCX register

A Stitch In Time Saves Nine: A Stitch In Time Saves Nine: A Case Of Multiple OS Vulnerability; Rafal Wojtczuk; BlackHat, 2012
Model Checking to Find Vulnerabilities in an Instruction Set Architecture; Bradfield et al; HOST'16

Two Different Specifications for SYSRET

AMD SYSRET

HW transits from kernel mode to user mode

Restore the userspace next-PC from the RCX register

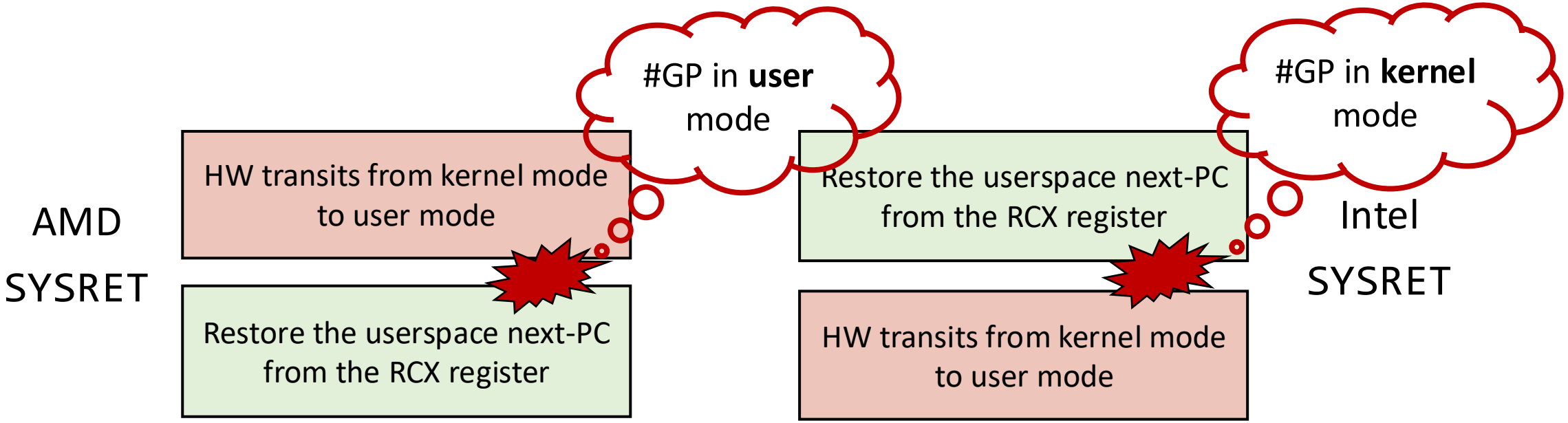
Intel SYSRET

Restore the userspace next-PC from the RCX register

HW transits from kernel mode to user mode

Order is flipped

SYSRET Vulnerability



How to trigger the exception?

If RCX holds a non-canonical address, the SYSRET will generates a #GP (general protection fault)

Canonical means that given 48-bit virtual address space, the high 16 bits (bits 63-48) of a virtual address have same value as bit 47.

How SYSRET is used in kernel code?

- What do we do before we transition from kernelspace to userspace?

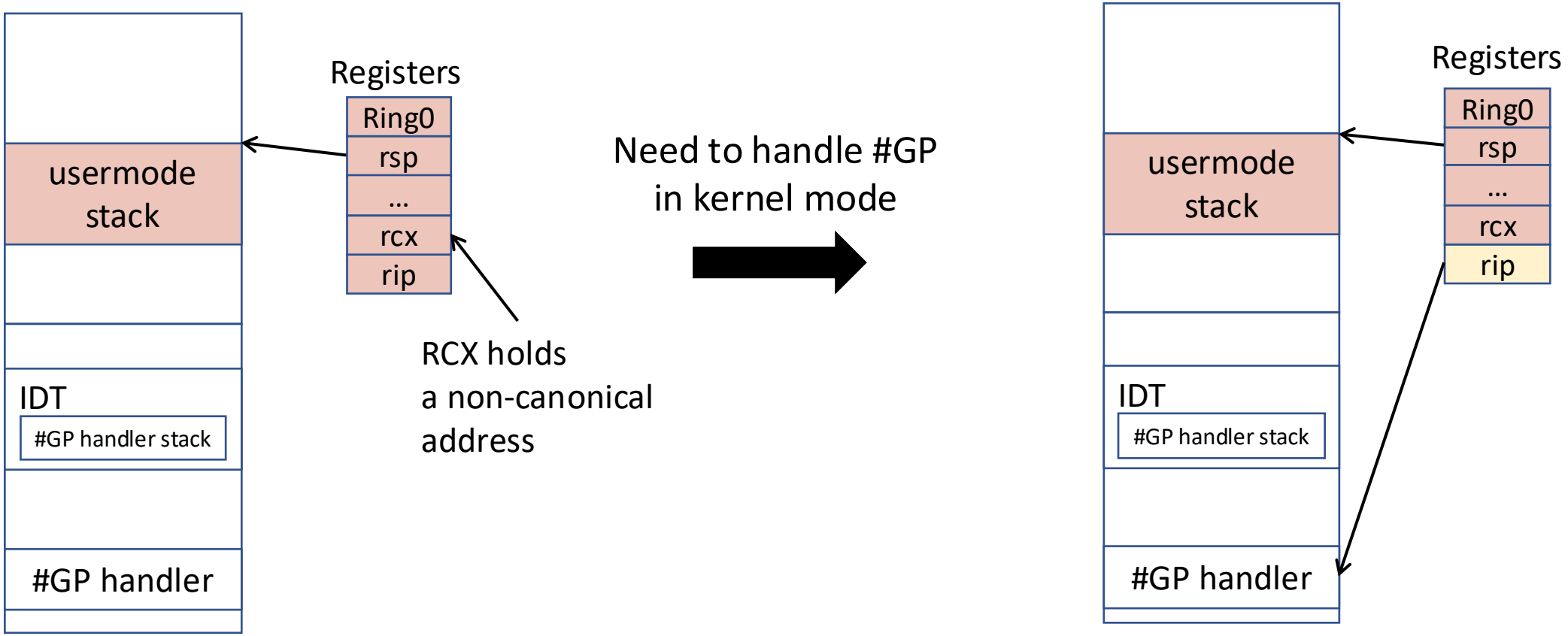
At this point,
all the registers are
user-controlled
(attacker-controlled)

```
movq RCX(%rsp), %rcx
movq RIP(%rsp), %r11
cmpq %rcx, %r11 /* SYSRET requires RCX == RIP */
jne swapgs_restore_regs_and_return_to_usermode

...
/* populate all the registers using data from userstack */
sysret
```

https://elixir.bootlin.com/linux/latest/source/arch/x86/entry/entry_64.S

SYSRET Attack on Intel Processors



Before executing SYSRET, all registers have been restored using usermode context

Assume rip points to kernel stack and start using it --> can **overwrite kernel data**

Longtime Intel x86 OS Bug

[PATCH] x86_64: Wh

Intel EM64T CPUs hanc
from AMD CPUs.

The exception is repo
This leads to the ker
with the wrong GS bec
on this instruction.

This version of the p
version fixed.

This is CVE-2006-074

Thanks to Ernie Petrides and Asit B. Mallick for analysis and initial patches.

2012-07-05

CVE-2012-0217: Intel's sysret Kernel Privilege Escalation (on FreeBSD)

By iZsh

Filed under vulnerability exploit FreeBSD

CVE-2012-0217 Detail

MODIFIED

This vulnerability has been modified since it was last analyzed by the NVD. It is awaiting rea changes to the information provided.

Description

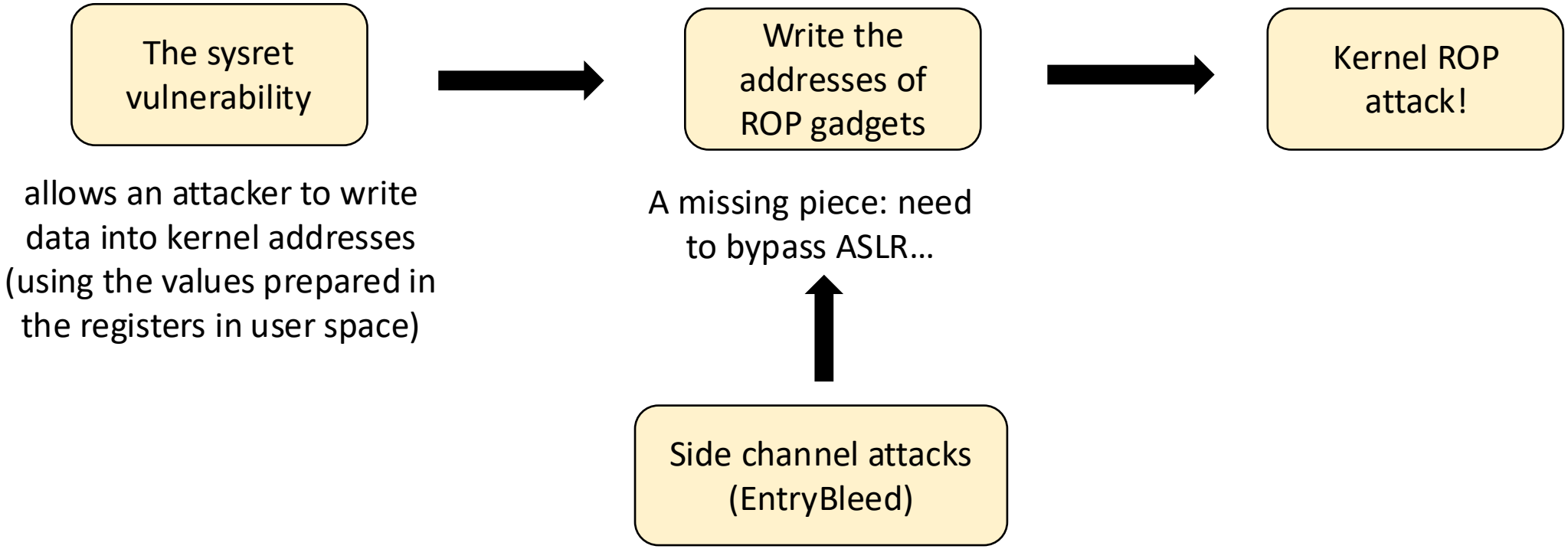
The x86-64 kernel system-call functionality in Xen 4.1.2 and earlier, as used in Citrix XenServe Oracle Solaris 11 and earlier; illumos before r13724; Joyent SmartOS before 20120614T184600Z; FreeBSD before 9.0-RELEASE-p3; NetBSD 6.0 Beta and earlier; Microsoft Windows Server 2008 R2 and R2 SP1 and Windows 7 Gold and SP1; and possibly other operating systems, when running on an Intel processor, incorrectly uses the sysret path in cases where a certain address is not a

CVE-2014-4699: Linux Kernel ptrace/sysret vulnerability analysis

by Vitaly Nikolenko

Posted on July 21, 2014 at 6:52PM

Exploiting Sysret on Linux in 2023



https://www.willsroot.io/2023/08/sysruption.html

Intel claims “it’s not a bug, it’s a feature”

Description

SYSRET is a companion instruction to the SYSCALL instruction. It returns from an OS system-call handler to user code at privilege level 3. It does so by loading RIP from RCX and loading RFLAGS from R11.[1] With a 64-bit operand size, SYSRET remains in 64-bit mode; otherwise, it enters compatibility mode and only the low 32 bits of the registers are loaded.

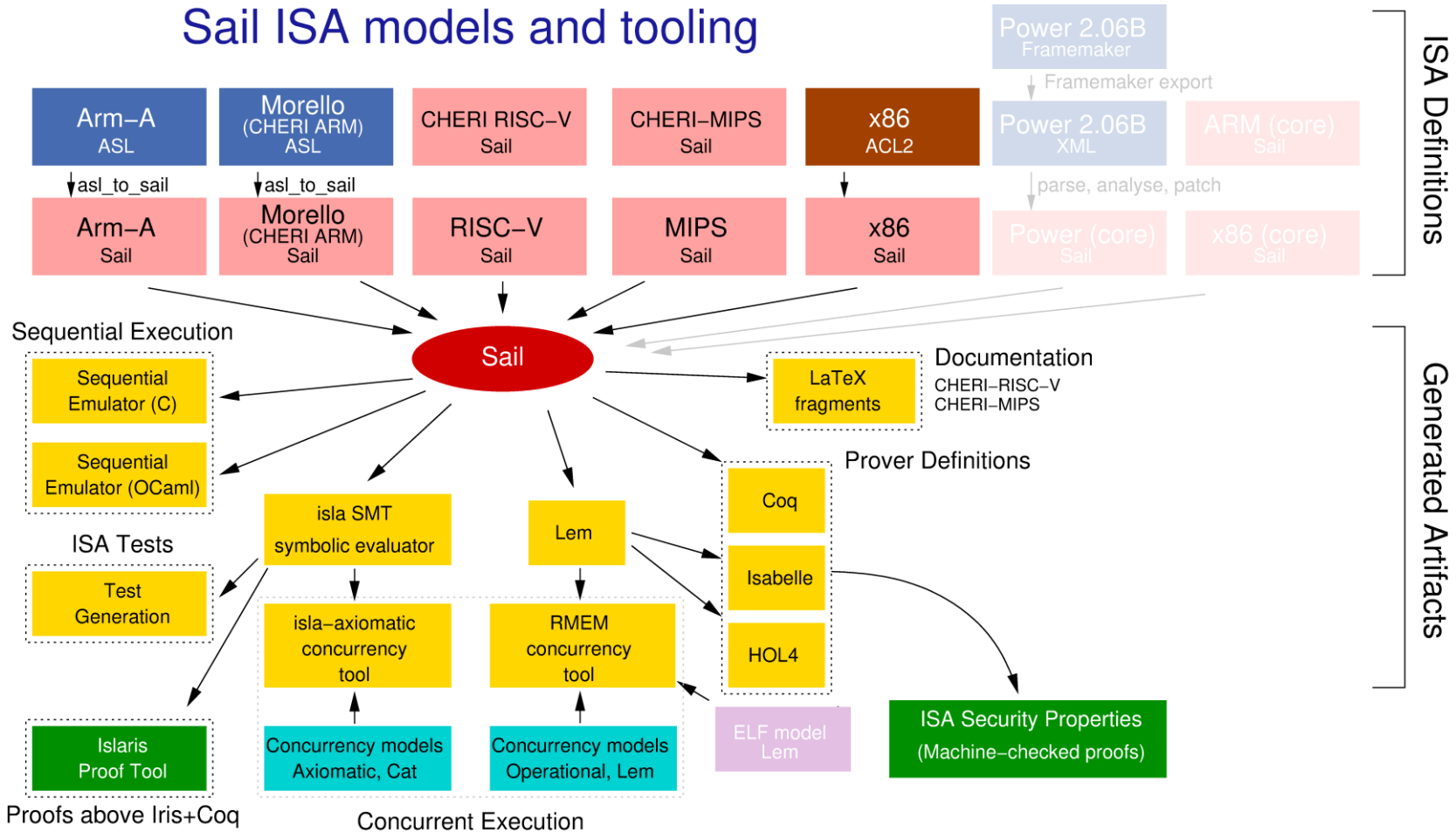
Intel claims that this vulnerability is a software implementation issue, as their processors are functioning as per their documented specifications. However, software that fails to take the Intel-specific SYSRET behavior into account may be vulnerable.

https://www.kb.cert.org/vuls/id/649219

Who to blame?

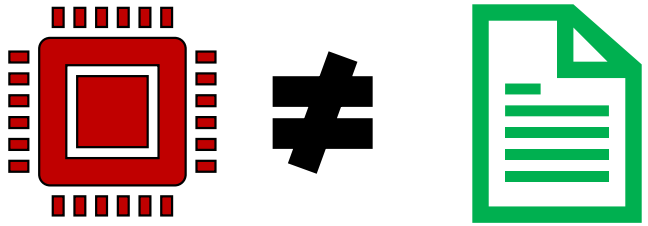
- Intel claims it is not an errata
  - *Errata are design defects or errors that may cause … behavior to deviate from published specifications*.
  - This behavior is consistent with Intel's specification
  - So the problem is the specification is incorrect
- Intel SDM (software development manual) 3400 pages. We cannot assume the specification is always correct.
- Research question: how can we know the ISA specification is correct?
  - Some research efforts to verify ISA specification

The Sail ISA specification language

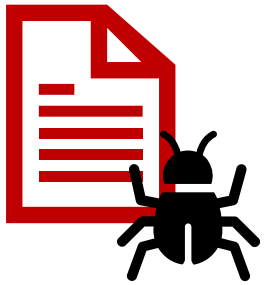


https://github.com/rems-project/sail

Summary



Implementation does not match specification (Errata)



Bugs in the specification

Underspecification

- Next: How to find hardware bugs?
  - Get ideas from the software

Software Bugs Hunting/Fixing

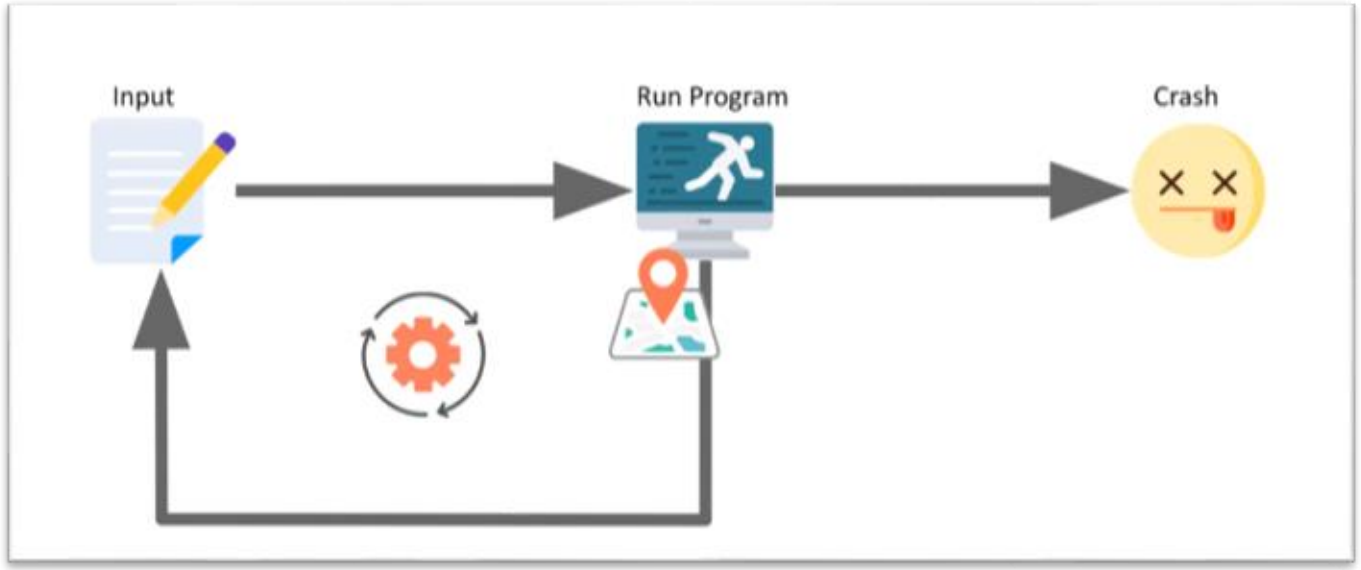
- Approach 1: Manual effort
  - Hire a lot of experts and stare at the code
  - Regression test → but need to be updated
- Approach 2: How about randomly generating test cases?
  - Fuzzing
- Approach 3: Formal verification
- Approach 4: Use LLM

Fuzzing

Fuzzing In A Nutshell

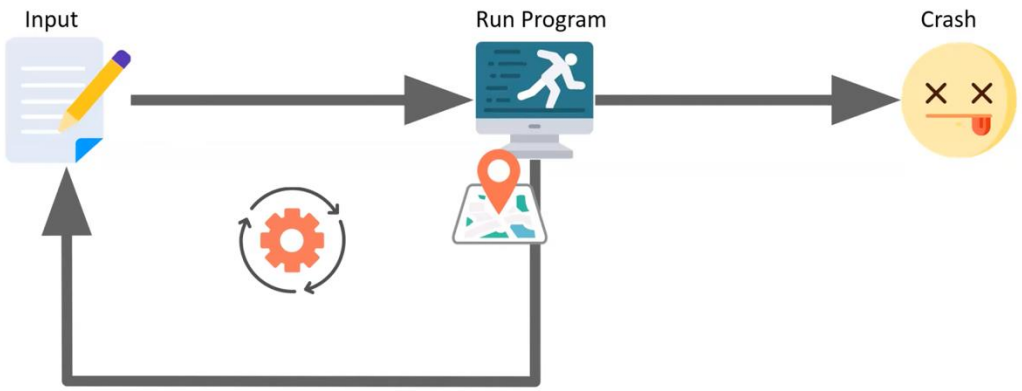
- Automatic generate test examples
- 1999, Alan Cox at University of Wales discovered a vulnerability in Linux kernel by simply running a proram generating random input and feed into the kernel
- Crash is generated by assertions/specifications

- Simple yet effective
- Industry standard for software



From Riding the Fuzzing Hype Train (RAID'21 Keynote)

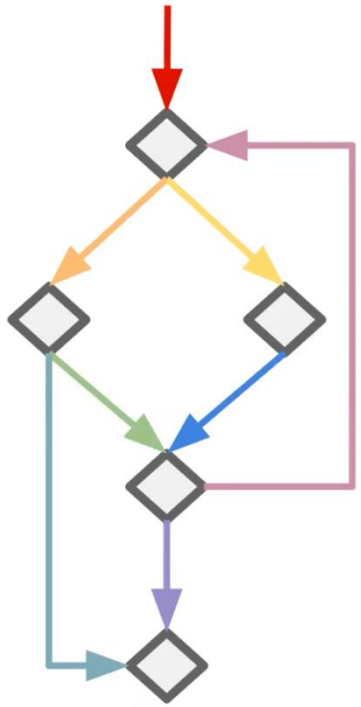
Fuzzing Components



- Q1: How to pick input?
  - Rely on random seeds
  - Sometimes need formatted inputs, e.g., PDF reader
- Q2: How to check whether the program under test generates the correct outcome?
  - Compare with specification's output
  - Assertions (Address Sanitizer, e.g., using ARM MTE)
- Q3: How to generate new test cases?
  - Heuristics for generating new tests => feedback loop for better efficiency

Types of Fuzzing

- Blackbox
- Greybox
- Whitebox



Collected coverage:

1	6	2	6	0	2	1	7

From Blackbox Fuzzing to Whitebox Fuzzing towards Verification; Patrice Godefroid; Microsoft Research

Fuzz the Hardware

Processor Input: Instruction Encoding

- There exist holes in instruction encoding:

31 – 25	24 – 20	19 – 15	14 – 12	11 – 7	6 – 0
funct7	rs2	rs1	funct3	rd	opcode
7	5	5	3	5	7
0 0 0 0 0 0 0	src2	src1	ADD/SLT[U]	dest	OP
0 0 0 0 0 0 0	src2	src1	AND/OR/XOR	dest	OP
0 0 0 0 0 0 0	src2	src1	SLL/SRL	dest	OP
0 1 0 0 0 0 0	src2	src1	SUB/SRA	dest	OP

The RISC-V Instruction Set Manual Volume I: Unprivileged ISA

What will happen if we try to execute an invalid instruction?

```
addi x0, x0, 0      # valid
# encoding: 0x00000013

Flip a bit: 0x00000013 -> 0x00000012
low 7 bits are the opcode: 0010011, which is invalid in RISC-V
```

Breaking the x86 ISA, Christopher Domas; Blackhat'17
https://www.youtube.com/watch?v=KrksBdWcZgQ

Example: Hidden Instructions

- There exist holes in instruction encoding:

31 – 25	24 – 20	19 – 15	14 – 12	11 – 7	6 – 0
funct7	rs2	rs1	funct3	rd	opcode
7	5	5	3	5	7
0 0 0 0 0 0 0	src2	src1	ADD/SLT[U]	dest	OP
0 0 0 0 0 0 0	src2	src1	AND/OR/XOR	dest	OP
0 0 0 0 0 0 0	src2	src1	SLL/SRL	dest	OP
0 1 0 0 0 0 0	src2	src1	SUB/SRA	dest	OP

The RISC-V Instruction Set Manual Volume I: Unprivileged ISA

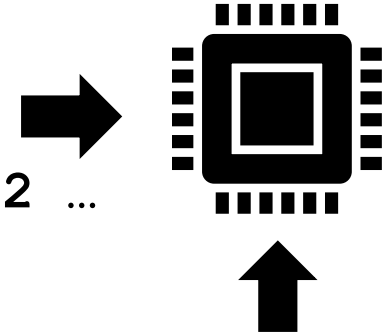
- Hidden instructions: secret instructions that give backdoor or powerful access to processor internals
- Secret processor functionality: Appendix H
- An example:
  - Pentium F00F bug, an invalid instruction freezes the cpu, discovered in 1997
  - A Ring 3 process can DOS (denial of service) a process
  - The invalid instruction encoding is: **`F0 0F C7 [C8-CF]`**

Breaking the x86 ISA, Christopher Domas; Blackhat'17
https://www.youtube.com/watch?v=KrksBdWcZgQ

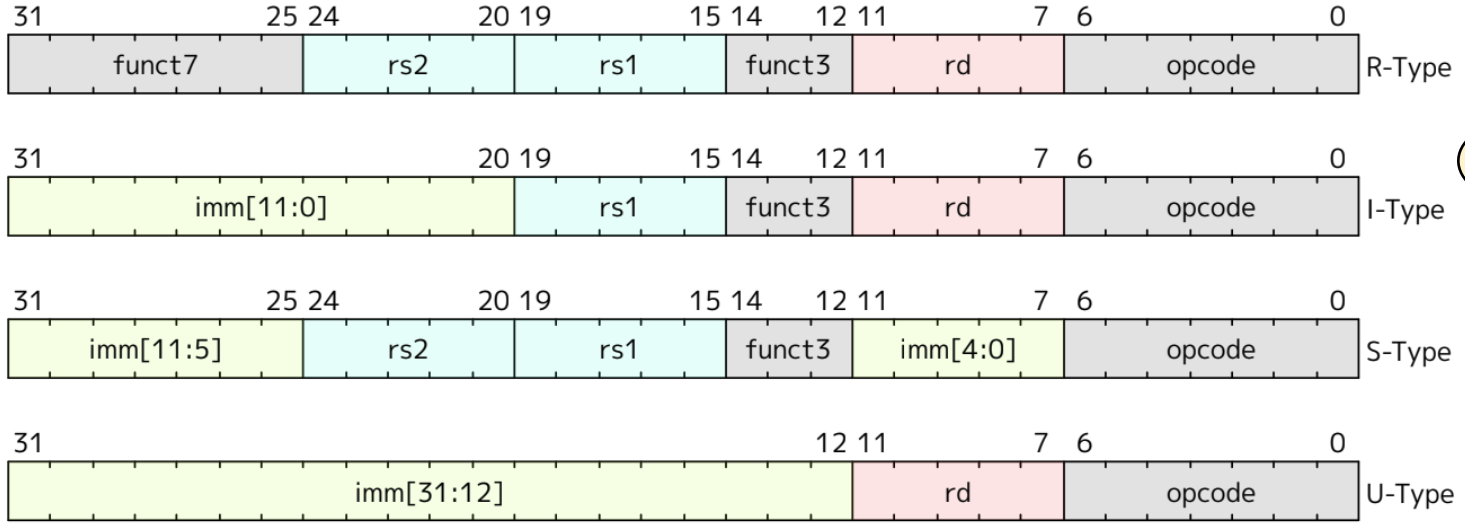
Search for Hidden Instructions

Instructions:

`0F 6A 60 6A 79 6D C6 02 …`



ISA specification:



Valid instructions (in spec)

Invalid instructions (#UD exception, invalid opcode)

Hidden instructions (not in spec, but can execute, no #UD exception)

How to capture this case?

More Hardware Fuzzing Examples

- Zenbleed: found a CPU bug via post-silicon fuzzing

```
movnti [rbp+0x0],ebx
rcr dh,1
sub r10, rax
rol rbx, cl
xor edi,[rbp-0x57]
```

```
movnti [rbp+0x0],ebx
sfence
rcr dh,1
lfence
sub r10, rax
mfence
rol rbx, cl
nop
xor edi,[rbp-0x57]
```

A randomly generated sequence of instructions, and the same sequence but with randomized alignment, serialization and speculation fences added.

https://lock.cmpxchg8b.com/zenbleed.html

Summary

- Hardware bugs
  - Deviate from specification (errata)
  - Incorrect and under specification

- Potential approaches to find hardware bugs
  - Manual analysis, testing
  - Fuzzing
  - Formal Verification (next lecture)

> *Program testing can be quite effective for showing the presence of bugs, but is hopelessly inadequate for showing their absence.*
>
> – Edsger Dijkstra